

MediFlow – Hospital Management System

Author

Name: Om Manish Makadia

Roll Number: 23f3001593

Email ID: 23f3001593@ds.study.iitm.ac.in

About: I am an enthusiastic learner in Artificial Intelligence and Data Science, currently pursuing a diploma level in online BS degree in Data Science & Applications from IIT Madras, alongside an offline BE degree in Artificial Intelligence & Machine Learning from BMS College of Engineering, Bengaluru.

Description

In this project, I have to develop a full-stack Hospital Management System (HMS) that allows Admins, Doctors, and Patients to manage all hospital-related activities through a unified web application. The system must support secure authentication for different user roles, prevent scheduling conflicts, and maintain complete patient treatment history. Additionally, I have to implement batch jobs such as daily appointment reminders, monthly doctor activity reports, and patient-triggered CSV exports of treatment history. I must also integrate caching, performance optimization, and validation across the system to ensure smooth operation.

“**MediFlow**” signifies a smooth, streamlined flow of medical operations—ensuring that hospital tasks like appointments, treatment records, and communication move seamlessly without delays or conflicts.

AI/LLM Use:

Usage Percentage – 8% (Bootstrap / CSS)

I have used AI/LLM (ChatGPT) in this project primarily as a learning and guidance tool to understand concepts, clarify doubts, and explore best practices. No code was copied from the AI; all implementation is original and written by me, based on my understanding. The AI's role was purely educational to help me understand the concepts. The only area of direct assistance was in custom CSS and Bootstrap, where I gave clear design requirements and later understood the purpose of each suggested element. Overall, AI/LLM was used to support conceptual understanding and guide implementation decisions, ensuring that all code was developed independently with full ownership, understanding and clarity.

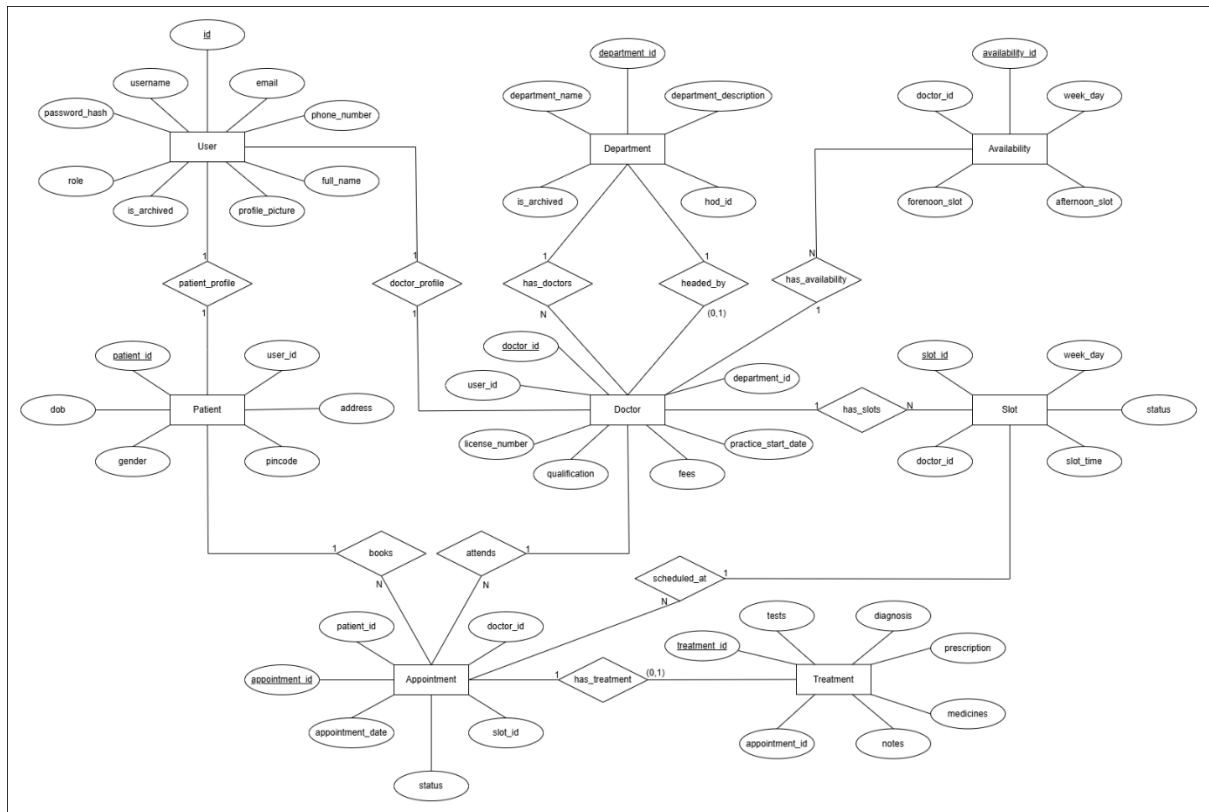
Technologies Used

Technology	Purpose
Vue.js	Used to build a reactive and component-based user interface for the application.
Vuex	Used for centralized state management to share data across multiple Vue components.
Vue Router	Used to handle client-side navigation and manage routes between pages in frontend.
JavaScript	Used to handle Axios API calls and implement utility functions for frontend logic.
CSS (Custom)	Used to apply minor custom styles and overrides not covered by Bootstrap.
Bootstrap	Used to quickly create responsive, mobile-friendly layouts using predefined styles.
SQLite	Used as the lightweight, file-based relational database to store all application data.
Flask	Used as the main backend framework to build RESTful APIs.
Flask-SQLAlchemy	Used as the ORM to interact with the SQLite database through Python models.

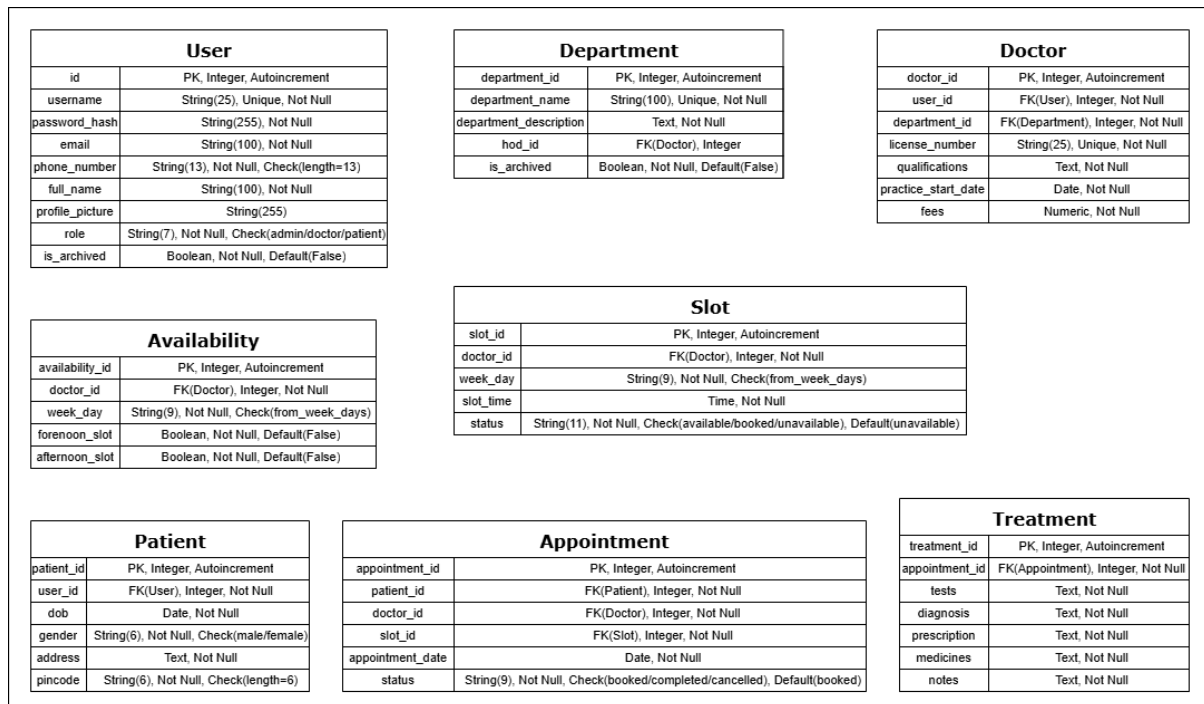
Flask- Bcrypt	Used to securely hash and verify user passwords.
Flask-JWT-Extended	Used to implement token-based authentication and role-based access control.
Flask-Caching	Used to improve API performance using cache storage to reduce repeated queries.
Flask-CORS	Used to enable secure cross-origin communication between the frontend and backend.
Flask-Mail	Used to send automated emails such as monthly reports or reminders from the backend.
Redis	Used for caching, background jobs, and task queue management.
Celery	Used to schedule and run asynchronous background tasks.
Werkzeug	Used to create custom HTTP exceptions.
ReportLab	Used to generate PDF reports such as monthly doctor activity reports.

DB Schema Design

ER Diagram (drawn using draw.io):



Relational Schema (drawn using draw.io):



Reasons behind the Database Design:

- Single User Table for All Roles (admin, doctor, patient):
A unified User table prevents data duplication and keeps authentication simple; roles such as admin, doctor, and patient are distinguished using the role field instead of creating separate login tables.
- Role-Specific Details Stored in Separate Tables (Doctor & Patient):
Instead of overloading the User table, role-specific information (doctor qualifications, patient demographics) is placed in separate Doctor and Patient tables, ensuring normalization and clean separation of concerns.
- Department as a Core Organizational Entity:
The Department table structures doctors and appointments logically; every doctor must belong to one department, reflecting real hospital workflow and allowing department-wise dashboards, filtering, and analytics.
- HOD Linked via Doctor Table (1:1 Relationship):
The hod_id field references a doctor, ensuring that only existing doctors can be assigned as Head of Department and that each department has exactly one HOD.
- Doctor Availability and Slots Separated for Flexibility:
Availability defines weekly availability (e.g., Monday forenoon), while Slot stores actual bookable time slots; this dual-table design supports both fixed weekly schedules and dynamic per-slot booking.

- Appointment Table Linking Patient, Doctor, and Slot:
The Appointment table acts as a multi-link join connecting doctor, patient, and slot, enabling full tracking of scheduled visits and preventing double-booking of time slots.
- One-to-One Treatment Mapping:
Each appointment may have exactly one treatment record, stored in the Treatment table; this avoids storing large text fields (diagnosis, prescription, notes) inside the Appointment table, keeping it lightweight.
- Status Fields to Track Lifecycle of Entities:
Attributes like `is_archived`, `status(slot)`, and `status(appointment)` are used to soft-delete or manage real-world states without losing historical data.
- Strong Data Integrity via Check Constraints:
Constraints enforce domain rules (e.g., valid weekdays, genders, role list, pin code length, slot statuses) to maintain clean, trustworthy data and prevent invalid inputs from reaching the database.

API Endpoints

Auth API:

Route	Method	Explanation
/api/auth/login	POST	Authenticates a user using username and password and returns JWT access & refresh tokens.
/api/auth/refresh	POST	Generates a new access token using a valid refresh token.

Admin API:

Route	Method	Explanation
/api/admin	GET	Fetches admin profile (cached).
/api/admin	PUT	Updates admin profile and clears related caches.
/api/admin/department	POST	Creates a new department.
/api/admin/department/< department_id>	GET	Retrieves full details of a department (cached).
/api/admin/department/< department_id>	PUT	Updates department details and clears related caches.
/api/admin/department/< department_id>	PATCH	Soft deletes (archives) a department.
/api/admin/departments	GET	Returns list of all departments (cached).
/api/admin/assign-hod/<department_id>/<doctor_id>	PUT	Assigns or updates the Head of Department.
/api/admin/doctor	POST	Creates a new doctor under a department.
/api/admin/doctors	GET	Retrieves list of all doctors (cached).
/api/admin/patients	GET	Returns list of all patients (cached).
/api/admin/appointments	GET	Returns all appointments for the admin panel (cached).
/api/admin/summary	GET	Provides dashboard summary statistics (cached).

Doctor API:

Route	Method	Explanation
/api/doctor/<id>	GET	Fetches doctor profile with department & user data (cached).
/api/doctor/<id>	PUT	Updates doctor profile, including optional profile picture.
/api/doctor/<id>	PATCH	Soft deletes (archives) a doctor.
/api/doctor/availability/<id>	GET	Retrieves doctor's weekly availability (cached).
/api/doctor/availability/<id>	PUT	Updates doctor availability for all weekdays.
/api/doctor/slot/<doctor_id>	GET	Gets availability slots for the doctor (cached).
/api/doctor/appointments/scheduled/<id>	GET	Fetches upcoming appointments of the doctor (cached).
/api/doctor/appointments/past/<id>	GET	Retrieves past completed appointments (cached).
/api/doctor/treatment/<appointment_id>	POST	Adds treatment details for a completed appointment.

Patient API:

Route	Method	Explanation
/api/patient	POST	Creates a new patient linked to a user.
/api/patient/<id>	GET	Retrieves patient profile with user details (cached).
/api/patient/<id>	PUT	Updates patient profile, including optional profile picture.
/api/patient/<id>	PATCH	Soft deletes (archives) a patient.
/api/patient/appointment/<id>/<doctor_id>	POST	Books a new appointment for a given doctor and patient.
/api/patient/appointment/<appointment_id>	PUT	Reschedules the appointment.
/api/patient/appointment/<appointment_id>	PATCH	Cancels or deletes an appointment.
/api/patient/appointments/scheduled/<id>	GET	Retrieves upcoming scheduled appointments (cached).
/api/patient/appointments/past/<id>	GET	Retrieves past completed appointments (cached).
/api/patient/treatments/<id>	GET	Fetches full treatment history for the patient (cached).
/api/patient/treatments/export/<id>	POST	Exports all treatments into a CSV file (background task).

Video

https://drive.google.com/file/d/1bbKoE0KHZFAOBiZj-qAze_opx5UiByAk/view?usp=drive_link